Lecture 21 · Part V · Extended notes

Recommender systems: from ratings to factors

Dataset: MovieLens 1M

Why these notes exist

Lectures 19–22 sit outside Géron, the course's primary text. For those four lectures *these notes are the primary source*, and they are examined on exactly the same terms as the textbook parts. Everything in the slide deck for this lecture is here, with the arguments written out at length rather than compressed onto a slide; the numbers are the ones the accompanying notebook prints, and every one of them is a measurement on the corpus named above rather than a figure quoted from a paper.

Contents

1 Retrieval, with the query removed **3**

2 The problem, and the data **3**

2.1 Why MovieLens, and what it is not . . . 3

2.2 The matrix, and what is not in it . . . 4

2.2.1 Not missing at random . . . 4

2.3 What "a user" means in the model . . . 4

2.4 What people actually give . . . 4

2.5 Why RMSE, and its quiet assumption . . . 5

2.6 The long tail . . . 5

3 Baselines: how much of a rating is the user, and how much the film **5**

3.1 The split, stated first . . . 6

3.2 The baselines . . . 6

3.3 The bias model, written out . . . 6

3.4 Cold start, named now . . . 7

4 Derivation: matrix factorisation, and the SVD it is not **7**

4.1 The low-rank hypothesis . . . 7

4.2 Lecture 6 already solved this — or did it . . . 7

4.3 So fill them in? . . . 8

4.3.1 Why filling fails . . . 8

4.4 Change the objective, not the data . . . 9

4.5 Hold one side fixed . . . 9

4.6 Or descend, one rating at a time . . . 10

4.7 Put the biases back . . . 10

4.8 How many factors? . . . 10

5 Neighbourhoods, and feedback nobody gave **11**

5.1 The other family . . . 11

5.2 Item–item similarity . . . 11

5.3 Why factorisation wins, and where it does not . . . 12

5.4 Explicit ratings are the easy case . . . 12

5.5 Implicit feedback has no negatives . . . 12

5.5.1 BPR: rank the pair, not the item . . . 13

6 What was built **13**

6.1 The distance travelled, in one number . . . 13

6.2 What a deployed recommender actually is . . . 13

6.3 The notebook . . . 14

7 Design exercise: an e-learning company's course recommender **14**

8 Exercises **16**

9 Summary **16**

1 Retrieval, with the query removed

	Retrieval	**Recommendation**
Input	a query and a corpus	a *user* and a catalogue
Output	a ranking	a ranking
Signal	the query text	what this user did before
Labels	judged by assessors, deliberately	a *by-product of use*

The output is the same shape, so Lecture 19's metrics apply unchanged. The *input* is what differs, and the last row is what makes this hard: nobody rated a film in order to help you.

The previous lecture put queries and documents in one space with a bi-encoder, fused its ranking with BM25's, and re-ranked the top of the list with a cross-encoder. Here the query disappears, and a history takes its place.

2 The problem, and the data

6,040 users. 3,706 films. 1,000,209 ratings on a 1–5 scale. Two questions, and they are not the same question.

- **Rating prediction** — what would this user give this film? Scored by RMSE. That is this lecture.
- **Top-*N* recommendation** — which ten films should we show? Scored by the ranking metrics of Lecture 19. That is Lecture 22.

They come apart, which is the subject of the next lecture: a model that predicts every rating within half a star can still recommend badly, because the films a user would rate highly are not the films they have not seen.

2.1 Why MovieLens, and what it is not

It has *explicit ratings*, which most systems do not have — but without them, rating prediction cannot be taught at all. It has *timestamps*, which is why this release and not the smaller one: half of Lecture 22 is the difference between splitting a history at random and splitting it in time. And a million ratings is large enough that the results are not noise and small enough to factorise on a free CPU.

And what it is not

A research dataset from a system that *asked people to rate films*. It is not a log of a production recommender, its users are self-selected, and its sparsity pattern is unlike a real catalogue's. Everything measured here transfers as a *method*, not as a number.

2.2 The matrix, and what is not in it

Users × films	22.4 million cells
Ratings present	1,000,209
Density	4.5%
Ratings per user, median	96
Ratings per film, median	124

95.5% of the matrix is missing, and this is the central fact of the lecture.

Missing is not zero

A cell is empty because the user never saw the film, or saw it and did not rate it, or was never shown it by whatever system was running at the time. Every one of those is a different meaning, and none of them is "disliked".

Not missing at random

Worse: the pattern of what is missing is *informative*. People rate films they chose to watch, and they chose them because they expected to like them. A horror film unrated by a user who hates horror is missing *because* they hate horror — which is a rating of a kind, and it is not in the data. Every model in this lecture trains on observed entries and is evaluated on observed entries, so none of them ever sees this.

That is the same shape as Lecture 19's pooling: the data records what someone chose to look at, and the choice was not random. In retrieval it made recall a lower bound; here it biases the training set itself.

2.3 What "a user" means in the model

p_u is a vector attached to an *account*, and an account is not a person.

- A household shares one account, so p_u averages several tastes into one point — and the average of two tastes is often nobody's.
- A person's taste changes; p_u is a single vector with no time in it, and MovieLens spans years.
- The same person on two devices is two users, with two independent vectors estimated from half the data each.

Nothing in the objective represents any of this. A model whose unit of analysis is wrong will be confidently wrong, and no amount of held-out RMSE reveals it.

2.4 What people actually give

Rating	1	2	3	4	5
Share	5.6%	10.8%	26.1%	34.9%	22.6%

Mean 3.58, heavily skewed upward: 57.5% of ratings are 4 or 5. People mostly rate things they liked, which means the scale is not being used as a scale — it is being used as "good", "very

good", and occasionally "I want you to know this was bad".

2.5 Why RMSE, and its quiet assumption

$$\text{RMSE} = \sqrt{\frac{1}{|\Omega_{\text{test}}|} \sum_{(u,i)\in\Omega_{\text{test}}} \left(r_{ui} - \hat{r}_{ui}\right)^2}$$

Squared error, so a two-star miss costs four times a one-star miss. That is defensible and worth stating: it says being badly wrong occasionally is worse than being slightly wrong often.

The quiet assumption is bigger. RMSE averages over the *observed* test ratings — films these users chose to watch and rate. It says nothing about the films they did not, which is precisely the set a recommender operates on. Lecture 22 replaces it for that reason. Today it is the right metric for "what would they rate it" and the wrong one for "what should we show".

Failure condition — RMSE on what is really a ranking task

RMSE scores how well you predict ratings a user actually gave. A recommender is judged on what it puts in front of a user for items they have not touched, so a model can win on RMSE and still rank the catalogue worse than a popularity list.

2.6 The long tail

Top films by rating count	Share of all ratings
top 1%	8.7%
top 5%	28.3%
top 10%	44.4%
top 20%	65.2%
Films with fewer than 20 ratings	17.9%

Two consequences to keep. First, a model can score well by learning popularity and nothing else — Lecture 22 measures exactly that. Second, the films where a recommendation would be *valuable* are the ones with least data.

The tail is where the value is

A recommendation is useful to the extent the user would not have found the item anyway. The top 1% of films hold 8.7% of ratings: recommending those is easy, scores well, and tells the user nothing they did not know. And 17.9% of films have under twenty ratings — exactly where every model here has least to work with.

Nothing measured in this lecture rewards usefulness. RMSE over held-out ratings, and even NDCG over held-out interactions, give full credit for a correct popular recommendation and the same credit for a correct obscure one. Lecture 22 tries.

3 Baselines: how much of a rating is the user, and how much the film

3.1 The split, stated first

90/10 on the ratings, at random, one seed, fixed before any model was fitted: 900,189 ratings to fit and 100,020 to test. Every number in this lecture uses that split.

Failure condition — a random split of ratings is not innocent

It puts a user's *future* ratings in the training set and their past in the test set, which no deployed system ever gets. It is the standard protocol for rating prediction, it is convenient, and it is optimistic. Lecture 22 measures exactly how optimistic.

3.2 The baselines

Every number needs a baseline. Fit on the training part of the split, and predict:

Predictor	RMSE
The global mean, for everyone	1.1185
This user's mean rating	1.0367
This film's mean rating	0.9826
Both, as biases	0.9109

A single number for everybody scores 1.1185. Knowing which film takes it to 0.9826; knowing who *and* which film takes it to 0.9109. No interaction between the two has been modelled yet — nothing so far knows that *this* user likes *this kind* of film.

3.3 The bias model, written out

$$\hat{r}_{ui} = \mu + b_u + b_i$$

Fitted by alternating closed forms, each regularised by the count, because a user with three ratings should not be given a large offset:

$$b_u = \frac{\sum_{i \in I_u} (r_{ui} - \mu - b_i)}{|I_u| + \lambda}.$$

That λ in the denominator is shrinkage — a ridge penalty, λ times a squared norm added to the objective, in a new place: with few observations, pull the estimate toward zero.

	Lowest	Highest
User offset b_u	−2.2156	+1.4690
Film offset b_i	−1.8545	+1.0546

The harshest user rates nearly 2.2 stars below the mean on everything and the most generous 1.5 above. That spread is larger than most of what the rest of this lecture will add.

Why this matters more than it looks

A recommender that ignores biases spends its capacity re-learning that some users are generous and some films are popular — effects with nothing personal in them. Modelling them explicitly and cheaply frees the interesting part of the model for the interesting part of the problem.

3.4 Cold start, named now

A user with no ratings has no p_u to solve for; a film with no ratings has no q_i. The factorisation has nothing to say about either, and the failure is total rather than gradual. A new user falls back to popularity, or you ask for a few ratings up front. A new item falls back to content: genre, cast, description, an embedding of the synopsis. A new *system* has neither, which is why every recommender starts life as a popularity list — and why a real one is always a hybrid of a factorisation and something that works without history. The fallbacks themselves are not examinable in detail.

Failure condition — a user with no history

Matrix factorisation learns a vector per user from that user's ratings. A user with none has no vector to learn, so the model returns the bias term and nothing else — and an offline metric computed on users who all have histories never contains that case.

4 Derivation: matrix factorisation, and the SVD it is not

4.1 The low-rank hypothesis

Assume tastes are simpler than the data: that there are d underlying factors — how much a film is a comedy, how dark, how old — and that a rating is how much the user wants each factor times how much the film has of it.

$$\hat{r}_{ui} = p_u \cdot q_i, \qquad p_u, q_i \in \mathbb{R}^d,$$

equivalently $\hat{R} = PQ^\top$ with rank $\leq d$. The parameter count falls from mn to $d(m+n)$, which at $d = 16$ is about a hundredth of the matrix.

It is a *hypothesis*, not a fact. Its justification is that it predicts held-out ratings better than the alternatives, and we check that rather than assert it.

4.2 Lecture 6 already solved this — or did it

The best rank-d approximation of a matrix in the Frobenius norm is its truncated SVD. Eckart–Young:

$$\min_{\operatorname{rank}(X)\leq d} \|R - X\|_F \;=\; \left\|R - U_d \Sigma_d V_d^\top\right\|_F.$$

So the answer looks as though it is already in our hands: take the SVD of the ratings matrix, keep d singular values, read off P and Q.

Failure condition — one condition, and it is not met

The Frobenius norm sums over *every* entry of R. To evaluate it — let alone minimise it — every entry must have a value, and 95.5% of ours do not.

Be precise about what the theorem says, because the derivation turns on it. It is a statement about *one given matrix* R; the norm sums over every entry with equal weight; and it says nothing about entries R does not have, because the question is not defined. The theorem is not wrong. It is being misused. Filling the matrix does not extend the theorem to our problem — it *changes our problem* into one the theorem covers, and the two problems have different answers.

4.3 So fill them in?

Two obvious choices, both actually tried, both measured on the same held-out ratings.

Rank-32 SVD of. . .	**RMSE**
the matrix filled with zeros	2.3274
the matrix filled with the global mean	0.9920
predicting the global mean for everyone	1.1185
the bias model	0.9109

Filling with zeros scores 2.3274 — more than twice the error of predicting one number for everybody, *from the mathematically optimal rank-32 approximation*.

Ratings run from 1 to 5, so a zero is not a low rating: it is off the scale. The approximation is dragged toward zero everywhere and the predictions for observed pairs come out far too low. Filling with the mean at least puts the fill inside the range, so the damage is bounded: 0.9920, merely worse than the bias model rather than catastrophic. Both are the same error — one is simply more visible, and the more visible error is the safer one to make.

Why filling fails

The SVD did exactly what it promised: the best rank-32 approximation to the matrix it was *given*. That matrix is 95.5% zeros, so the best approximation is mostly a good approximation of the zeros. The fill value is not data, and optimising against it spends the model on reproducing an assumption — here a strong one: zero means "this user would rate this film 0", on a scale that starts at 1. Filling with the mean is less absurd, and still worse than the bias model, which has $d = 0$.

The transferable error

Imputing missing values to make a method applicable makes the method answer a different question. It is the same mistake as filling a missing feature with a column mean and then reporting the model as if the data had been complete.

4.4 Change the objective, not the data

The problem was never the SVD. It was that we asked for a fit to *every* entry. So ask only for what we can observe:

The factorisation objective

$$\min_{P,Q} \sum_{(u,i)\in\Omega} \left(r_{ui} - p_u \cdot q_i\right)^2 \; + \; \lambda\left(\|P\|_F^2 + \|Q\|_F^2\right)$$

- Ω — the observed entries, *and only those*. This is the whole departure from the SVD.
- $p_u \cdot q_i$ — a rank-d model written entrywise rather than as a matrix product, because the matrix product is over cells we do not have.
- λ — without which a user with three ratings gets a perfect fit and no information.

Three symbols, three decisions. If you can reconstruct why each is there, you have the derivation.

The unobserved entries are now *absent* from the objective rather than assigned a value. Nothing is imputed and nothing is assumed about what a user would have said. And in doing so we have left the world where the SVD applies: this objective has no closed form, no orthogonality and no unique solution — it is not an eigenproblem any more.

4.5 Hold one side fixed

The objective is not convex in P and Q together. But fix Q, and every term involving p_u is a quadratic in p_u alone:

$$\min_{p_u} \sum_{i\in I_u} \left(r_{ui} - p_u \cdot q_i\right)^2 + \lambda\|p_u\|^2,$$

which is a *ridge regression*, with the films this user rated as the rows of the design matrix and their ratings as the target. It is separate for each user: no user's solution depends on another's.

The ALS half-step

Write Q_u for the $|I_u| \times d$ matrix of the films user u rated and r_u for their ratings. Setting the gradient to zero,

$$p_u = \left(Q_u^\top Q_u + \lambda I\right)^{-1} Q_u^\top r_u.$$

The normal equation of Lecture 2 with a ridge term λI added to the Gram matrix. The system is $d \times d$ — 16×16 here — regardless of how many films the user rated.

Note where the missing entries went: Q_u contains *only the films this user rated*. Nothing was imputed; the unobserved entries simply never enter the system.

By symmetry, fixing P makes each q_i a ridge regression over the users who rated film i. So: fix Q and solve for every p_u; fix P and solve for every q_i; repeat.

Proposition 4.1. *Each half-step solves its subproblem exactly, so neither can increase the objective. The objective is bounded below by zero, so the sequence of objective values converges.*

It converges to a *local* minimum: monotone descent is not global optimality, and a different initialisation gives a different answer. That is the price of changing the objective, and it was paid knowingly.

4.6 Or descend, one rating at a time

For each observed rating, take the error and step both vectors:

$$e_{ui} = r_{ui} - \hat{r}_{ui}, \qquad p_u \mathrel{+}= \eta\,(e_{ui} q_i - \lambda p_u), \qquad q_i \mathrel{+}= \eta\,(e_{ui} p_u - \lambda q_i).$$

ALS parallelises across users and each step is exact, but needs a $d \times d$ solve per user. SGD touches one rating at a time, so it suits implicit data and streams, but needs a learning rate.

Same objective, different route. At $d = 32$, ALS reaches 0.8733 and SGD 0.8615.

4.7 Put the biases back

The factorisation models the *interaction*. The parts that are not interactions — a generous user, a popular film — should not be spent on it:

$$\hat{r}_{ui} = \mu + b_u + b_i + p_u \cdot q_i.$$

Fit the biases first, then factorise what they leave behind.

Bias model alone ($d = 0$)	0.9109
Factorisation, $d = 8$, with biases	0.8602
Factorisation, $d = 8$, without biases	0.8590

The biases alone close 80.4% of the distance from the global mean to the full model. *Most of a rating is not personal.*

An honest wrinkle

At $d = 8$ the model *without* explicit biases scores slightly better than the one with them. The textbook claim is that biases help; here they do not, and the reason is visible in the algebra — a factor dimension can hold a constant, so a rank-8 factorisation can represent the biases itself, using two of its eight dimensions to do it.

What is still true: biases are worth having because they are cheap, stable and interpretable, they help most at low rank, and they let the factors be about interaction. Not because they always lower RMSE — and the difference here, 0.0012, is small enough that reporting it as a finding either way would be overreaching.

4.8 How many factors?

d	1	2	4	8	16	32	64
Validation RMSE	0.8888	0.8811	0.8680	0.8591	0.8623	0.8798	0.9056

The validation ratings are 10% of the *training* ratings, held out; the models in the sweep are fitted on the rest. Best at $d = 8$, and rising again by $d = 64$. A U-curve: more capacity fits the training ratings better and the held-out ratings worse. The rank is a capacity parameter and it is chosen the way every capacity parameter in this course is chosen — by measurement on data the model did not see, and not on the test set it is reported on. Refitted at $d = 8$ on all the training

ratings, the test set is read once: 0.8602. Had the rank been chosen on the test ratings themselves, the reported number would have been the minimum of seven noisy estimates, and so biased downwards.

Even $d = 1$ scores 0.8888 on validation, already better than the bias model there (0.9090). One dimension of taste is worth something.

5 Neighbourhoods, and feedback nobody gave

5.1 The other family

Before factorisation there was a simpler idea, and it is still deployed: predict from similar things. **User-based** finds users who rated as you did and averages their ratings for the film in question; **item-based** finds films rated as this film was and averages this user's ratings of them.

Item-based is what gets deployed, for a reason that is operational rather than statistical: there are usually far fewer items than users, item–item similarities change slowly, and they can be computed offline and cached.

5.2 Item–item similarity

$$s_{ij} = \frac{\tilde{r}_i \cdot \tilde{r}_j}{\|\tilde{r}_i\| \, \|\tilde{r}_j\|}$$

— the cosine between two films' rating columns, where the tilde matters: the columns are *bias-adjusted* first, $\tilde{r}_{ui} = r_{ui} - \mu - b_u - b_i$. Without that, every popular film is similar to every other popular film, because both columns are full of fours.

That is the same trap as an unnormalised dot product in Lecture 20 and as term frequency without idf in Lecture 19: subtract what is true of everything before measuring what is specific.

$$\hat{r}_{ui} = \mu + b_u + b_i + \frac{\sum_{j \in N_k(i)} s_{ij} \, \tilde{r}_{uj}}{\sum_{j \in N_k(i)} s_{ij}}$$

Method	RMSE
item–item, $k = 10$	0.9376
item–item, $k = 20$	0.9171
item–item, $k = 50$	0.8867
factorisation, $d = 8$	0.8602

The best neighbourhood model does not reach the factorisation, and still beats the bias model.

5.3 Why factorisation wins, and where it does not

	Neighbourhood	Factorisation
Uses	direct co-rating evidence	a global low-rank structure
Sparse items	few co-ratings, poor estimate	borrows strength from everything
Explainable	"because you liked X"	a dot product of latent vectors
New rating	update a cached similarity	refit, or fold in approximately

Row three is why neighbourhood methods survive: a recommendation a user can be told the reason for is worth points a metric does not award. And in practice the two are combined — exactly as BM25 and dense retrieval were combined in Lecture 20, and for the same reason.

5.4 Explicit ratings are the easy case

Explicit	Implicit
a rating, deliberately given	a click, a play, a purchase, a dwell time
scarce — most users rate nothing	abundant — every interaction is one
a scale, with a middle	presence or absence
a stated preference	a revealed one, plus the interface

MovieLens is explicit, which is why it can be used to teach rating prediction at all. It is not representative: your first recommender will almost certainly have no ratings in it.

5.5 Implicit feedback has no negatives

A user watched film A. What does that say about film B, which they did not watch? They disliked it — a true negative. Or they have not seen it yet — possibly the best recommendation available. Or they were never shown it, and that was the old system's choice rather than theirs.

The whole difficulty

In explicit data the missing entries are excluded from the objective, and we saw exactly what happens when they are not. In implicit data the *only thing that exists* is the positives, so an objective over observed entries alone is trivially satisfied by predicting "yes" for everything.

The response taken here is to never predict a value at all, and require only that an observed item outranks an unobserved one for that user.

BPR: rank the pair, not the item

Bayesian personalised ranking

$$\max \sum_{(u,i,j)} \log \sigma\big(\hat{r}_{ui} - \hat{r}_{uj}\big) \; - \; \lambda \|\Theta\|^2,$$

with i observed for u and j sampled from the unobserved.

- Only the *difference* is modelled, so no absolute score is ever claimed — and none was ever observed.
- j is *sampled*, not enumerated, which is what makes it affordable.
- The logistic of a score difference is the pairwise logistic loss of Lecture 4, and the sampling is the negative sampling of Lecture 20.

And it inherits Lecture 20's trap exactly: a sampled "negative" may be an item the user would love. *Being unobserved is not evidence of dislike.*

6 What was built

6.1 The distance travelled, in one number

	RMSE	Distance closed
Global mean	1.1185	0%
Bias model	0.9109	80.4%
Item–item, $k = 50$	0.8867	89.7%
Factorisation, $d = 8$	0.8602	100%

Two arithmetic means and a shrinkage parameter get you most of the way; the derivation, the alternating solves and eight latent dimensions get the rest. That ratio is not an argument against the model. It is an argument for always fitting the baseline first, so that you know which part of your result is the idea and which part is arithmetic.

6.2 What a deployed recommender actually is

1. **Candidates**: catalogue → hundreds. Factorisation, popularity, co-visitation, recency — several sources, unioned.
2. **Rank**: hundreds → tens. A richer model with features the first stage cannot afford.
3. **Policy**: diversity, freshness, business rules, exploration.

The same three stages as Lecture 19's search pipeline, for the same reason: the accurate model does not scale to the catalogue. What we built today is stage 1. Stage 3 has no equation in it and decides more of what a user sees than stages 1 and 2 together.

6.3 The notebook

What to change

Set the regularisation to zero and re-run the rank sweep. Watch where the U-curve moves, and which rank stops being the best one. And reproduce the zero-filled SVD figure yourself — it is the most surprising number in Part V and it takes four lines.

7 Design exercise: an e-learning company's course recommender

The lecture closes with a design exercise in the manner of an ML system-design interview, on a product with the shape of data this model suits.

The task. A company sells a subscription to a catalogue of video courses, and its product team wants a row of courses "for you" on every learner's home screen. Behind it the system must estimate, for each (learner, course) pair, the rating the learner would give the course on the app's five-star scale, from what they and everyone else have rated so far. The business measures enrolments from the row, completions, and whether the subscription is renewed. Given: the ratings learners left on courses they finished, with timestamps; enrolments and completions for everyone; each course's title, topic and description. Hundreds of thousands of learners, thousands of courses, a matrix almost entirely missing, newcomers every day. The output feeds the "for you" row and the team that commissions new courses.

1. **Frame it.** Two questions, not one: what would this learner rate this course, scored by RMSE on held-out ratings; and which ten courses go in the row, a ranking over courses the learner has not touched, scored by ranking metrics. This exercise builds the first. RMSE charges a two-star miss four times a one-star miss, which says being badly wrong occasionally is worse than slightly wrong often — defensible, and worth stating to the product team; the first baseline is the global mean for everybody. *Pitfall:* reporting RMSE to the product team as if it measured the row. RMSE averages over ratings the learner chose to give on courses they chose to take, and the row is chosen from the courses they did not; a model within half a star on every held-out rating can still fill the row worse than the most popular courses, and the release is judged on enrolments from the row. Keep the two questions apart, and measure the row on untouched courses with the ranking metrics.

2. **The matrix.** Learners by courses, a cell holding a rating where one was given; most subscribers rate nothing, a rater rates a handful of the courses they finished, and the matrix is a few per cent full at best. An empty cell means never enrolled, or enrolled and never rated, or never shown by the old home screen — three meanings, none of them "disliked". What exists in abundance is implicit — enrolments, lessons watched, completions — and a click is not a rating and has no negative. *Pitfall:* missing is not zero, and not at random. A learner who dislikes statistics never enrols in the statistics course, so its cell is empty *because* they dislike it — a rating of a kind that is not in the data, and every model here trains and is scored on observed entries, so none of them sees it. Write the three meanings into the data team's description, and never let a method assign them a value.

3. **The split.** Hold out a share of the ratings for the test, at random, one seed, fixed before any model exists, and from the training ratings hold out a further share as validation ratings, on

which every choice — rank, regularisation, shrinkage — is made; the test ratings are read once. The timestamps allow a split in time, which is what the shipped system faces, so say which split a number came from. *Pitfall:* a random split of ratings is the standard protocol and it is optimistic: it puts a learner's later ratings in the training set and their earlier ones in the test set, which no deployed system ever gets. Acceptable for rating prediction if stated; not acceptable as a promise to the business about what the row will achieve.

4. **Baselines.** $\hat{r}_{ui} = \mu + b_u + b_i$: the global mean, this learner's offset, this course's offset, fitted by alternating closed forms, each regularised by the count. The λ in the denominator is shrinkage: a learner with three ratings is pulled towards zero offset, a course rated by thousands is barely moved. Expect the biases alone to close most of the distance from the global mean to the final model; most of a rating is not personal. *Pitfall:* skipping the baseline and crediting the factorisation with what two arithmetic means and a shrinkage parameter bought; or fitting it unshrunk, so that a course launched the week before with one five-star rating gets the largest offset in the catalogue, every home screen fills with courses rated once, and the commissioning team reads that as demand. Fit the bias model first, shrunk, and report every later model as what it adds beyond it.

5. **The model.** $\hat{r}_{ui} = \mu + b_u + b_i + p_u \cdot q_i$: a few latent factors — how practical a course is, how mathematical — and a rating is how much the learner wants each times how much the course has of it. The objective sums the squared error over the observed cells only, with a ridge penalty on both vectors; nothing is imputed. Fit by ALS — a ridge regression per learner, then per course, alternated — or by SGD one rating at a time when the matrix outgrows one machine's memory. *Pitfall:* filling the empty cells and taking the SVD, because the SVD is the best low-rank approximation — of the matrix it is given, which is almost entirely the fill value. Zeros are off a scale that starts at one and score worse than the global mean for everybody; the mean is less absurd and still loses to the bias model. Change the objective, not the data.

6. **The rank.** Sweep the rank from one upwards on the training ratings and score each on the validation ratings: a U-curve, falling as the factors capture taste and rising as they fit the noise. Take the bottom of the U, refit at that rank on all the training ratings, and read the test ratings once; the regularisation moves the curve, and at zero a learner with three ratings and a dozen factors gets a perfect fit and no information. *Pitfall:* choosing the rank on the test ratings. The number in the release review is then the minimum of several noisy estimates and is biased downwards, and the gain evaporates on live learners. The rank is a capacity parameter, chosen on data the model did not see and not on the set it is reported on.

7. **Cold start.** A learner with no ratings has no p_u to solve for and a course with no ratings has no q_i: the factorisation returns the bias term and nothing else, and the failure is total. A new subscriber falls back to popularity, or is asked for a few ratings at sign-up; a new course falls back to content — topic, description, an embedding of the syllabus — so the shipped system is a hybrid, and the switch between its halves is a rule the product team must write down. *Pitfall:* the offline RMSE never contains the case, because it is computed on learners and courses that all have ratings, so a model that fails totally on every newcomer scores the same as one that does not — and newcomers are the subscribers deciding whether to stay past the trial. Hold out some learners and some courses entirely, and report what the fallback achieves on them beside the headline number.

8. **In production.** Release behind an A/B test read on enrolments from the row and on completions, against the popular row; refit nightly as ratings arrive, with the on-call engineer watching catalogue coverage and the share of newcomers on the fallback. A predicted rating is what this learner would give if they took the course, learned from courses people chose: not a probability they will enrol, and not usefulness. For subscribers who never rate, the signal is implicit, and an objective over observed entries alone is satisfied by predicting "yes" everywhere, so rank pairs instead — an enrolled course above a sampled unenrolled one. *Pitfall:* popularity, and a sampled "no". A row of the ten most popular courses scores respectably on every metric here and tells the learner nothing new, which the renewal rate reports months later; and a sampled negative may be the course the learner would have loved, because unenrolled is not evidence of dislike. Report coverage beside the error, and read the implicit objective as a ranking, never as a prediction that anyone said no.

9. **Ship it.** Ratings log → the three meanings of an empty cell written down → test held out, validation carved from the training half → biases with shrinkage → factorisation of the observed entries → rank at the bottom of the validation U → fallbacks for newcomers → test read once, coverage beside it → A/B test on enrolments, nightly refit, monitor.

8 Exercises

The five set at the end of this lecture's deck, in the style of the written paper. The solutions are on Lecture 22's deck and in the exercise book.

1. State why the truncated SVD cannot be applied directly to a ratings matrix, naming the condition of Eckart–Young that fails. [5]

2. A rank-32 SVD of the zero-filled matrix scores worse than predicting one number for everybody. Explain how the optimal approximation can lose to a constant. [5]

3. Fix Q and write the subproblem ALS solves for one user's p_u. Identify it as a familiar estimator. [5]

4. Validation RMSE is lowest at $d = 8$ and rises again by $d = 64$. State what this says about the rank, and on which ratings it must be chosen. [4]

5. Implicit feedback has no negatives. State why an objective over observed entries alone fails, and how BPR's objective avoids the problem. [5]

9 Summary

Recommendation is retrieval with the query replaced by a history, and the labels are a by-product of use rather than a measurement. 95.5% of the matrix is missing, and missing is not zero: filling it and taking the SVD scores 2.3274, worse than predicting one number for everybody. Restrict the objective to observed entries and you leave the SVD behind — no closed form, no uniqueness, and a model that works. ALS is a ridge regression per user; SGD is one update per rating; same objective, different route, comparable answers. And most of a rating is not personal: the biases

alone reach 0.9109 against the full model's 0.8602. Implicit feedback has no negatives, which is why BPR ranks pairs instead of predicting values. The closing design exercise built an e-learning company's course recommender in nine steps — rating prediction kept apart from the row, the three meanings of an empty cell written down, validation ratings carved from the training half, biases shrunk by the count, a factorisation of the observed entries rather than a filled SVD, the rank at the bottom of the validation U, fallbacks for newcomers measured on their own, and an A/B test on enrolments with coverage beside the error — each step named for the pitfall it avoids.